

**LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 5 ACARA 9**

SEMESTER 3



**Tugas :
Service Layer, Validasi, Session,
dan Business Logic**

**Disusun Oleh:
Uaryl Engno Laurenca (E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. PENDAHULUAN

Pada aplikasi sederhana, Controller masih dapat menangani validasi dan proses penyimpanan data secara langsung. Namun, seiring bertambahnya kebutuhan bisnis seperti memastikan NIM wajib diisi, NIM tidak boleh duplikat, email berformat valid, dan program studi yang dipilih benar-benar tersedia — Controller akan menjadi sangat panjang dan sulit dipelihara apabila seluruh aturan tersebut dituliskan langsung di dalamnya.

Service Layer mengatasi masalah ini dengan menempatkan seluruh logika bisnis pada sebuah class Service tersendiri, sehingga Controller hanya bertugas menerima request, meneruskannya ke Service, lalu menentukan respons (redirect atau tampilkan error). Pada praktikum Acara 13 ini, dilakukan refactoring proses tambah dan ubah data mahasiswa pada project Sistem Informasi Akademik, dengan memindahkan seluruh validasi ke `MahasiswaService`, serta menerapkan mekanisme Flash Message berbasis session untuk menampilkan umpan balik kepada pengguna.

B. TUJUAN PRAKTIKUM

1. Membuat class `MahasiswaService` sebagai tempat logika bisnis proses tambah dan ubah data mahasiswa.
2. Menyuntikkan (*inject*) `MahasiswaRepository` dan `ProdiRepository` ke dalam `MahasiswaService` melalui constructor.
3. Memindahkan seluruh proses validasi data mahasiswa dari Controller ke `MahasiswaService`.
4. Membuat method `create()` pada Service untuk menangani proses penambahan mahasiswa.
5. Membuat method `update()` pada Service untuk menangani proses perubahan data mahasiswa.
6. Memastikan Controller tidak lagi melakukan validasi maupun query database secara langsung.
7. Menerapkan mekanisme Flash Message menggunakan `$_SESSION`.
8. Menampilkan pesan yang sesuai ketika data berhasil ditambahkan, berhasil diubah, gagal disimpan, atau NIM sudah terdaftar.
9. Memastikan Flash Message dihapus dari session setelah ditampilkan satu kali.
10. Menguji seluruh proses melalui browser.

C. DASAR TEORI

Service Layer adalah lapisan yang bertugas menangani logika bisnis aplikasi dan ditempatkan di antara *Controller* dan *Repository*. *Controller* bertugas menerima permintaan pengguna dan menentukan respons, sedangkan *Service* menangani aturan bisnis seperti memastikan NIM wajib diisi, NIM tidak boleh duplikat, email harus valid, serta program studi yang dipilih tersedia. Setelah seluruh aturan bisnis terpenuhi, *Repository* bertugas menjalankan proses penyimpanan atau pengambilan data dari database. Dengan pembagian tanggung jawab tersebut, *Controller* tetap sederhana,

Service menjadi pusat pengelolaan logika bisnis, dan *Repository* berfokus pada interaksi dengan database.

Flash Message dengan Session merupakan mekanisme untuk menampilkan pesan sementara kepada pengguna setelah suatu proses dilakukan. Pesan tersebut disimpan di dalam *session* dan hanya ditampilkan satu kali, kemudian dihapus setelah ditampilkan. Flash Message dapat digunakan untuk memberikan informasi mengenai keberhasilan maupun kegagalan suatu proses, seperti pesan “✓ Data mahasiswa berhasil disimpan” ketika proses berhasil atau “✗ NIM sudah terdaftar” ketika terjadi kesalahan. Dengan adanya Flash Message, pengguna dapat mengetahui hasil dari tindakan yang dilakukan pada aplikasi secara lebih jelas dan informatif.

D. HASIL DAN PEMBAHASAN

1. Pembuatan ProdiRepository

```
ACARA13 > app > Repositories > ProdiRepository.php
1  <?php
2
3  namespace App\Repositories;
4
5  use PDO;
6
7  class ProdiRepository
8  {
9      public function __construct(private PDO $pdo) {}
10
11     public function all(): array
12     {
13         $stmt = $this->pdo->query("SELECT * FROM prodi ORDER BY nama ASC");
14         return $stmt->fetchAll();
15     }
16
17     // Dipakai Service untuk memeriksa "program studi tersedia"
18     public function exists(int $id): bool
19     {
20         $stmt = $this->pdo->prepare("SELECT COUNT(*) FROM prodi WHERE id = :id");
21         $stmt->execute(['id' => $id]);
22         return (int) $stmt->fetchColumn() > 0;
23     }
24 }
```

Sebelum *MahasiswaService* dapat memvalidasi "program studi tersedia", dibuat terlebih dahulu *ProdiRepository* yang menyediakan method *all()* untuk daftar dropdown prodi, dan *exists()* untuk memeriksa apakah suatu *prodi_id* benar-benar ada di database.

2. Penambahan Method existsByNim() pada MahasiswaRepository

```
81 // Dipakai Service untuk cek "NIM sudah terdaftar"
82 public function existsByNim(string $nim, ?int $ignoreId = null): bool
83 {
84     $sql = "SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim";
85     $params = ['nim' => $nim];
86
87     if ($ignoreId !== null) {
88         $sql .= " AND id != :id";
89         $params['id'] = $ignoreId;
90     }
91
92     $stmt = $this->pdo->prepare($sql);
93     $stmt->execute($params);
94
95     return (int) $stmt->fetchColumn() > 0;
96 }
97
```

MahasiswaRepository ditambahkan satu method baru, `existsByNim()`, yang memeriksa apakah suatu NIM sudah terdaftar di database. Method ini menerima parameter opsional `$ignoreId`, agar saat proses **update**, NIM milik data itu sendiri tidak keliru dianggap sebagai duplikat.

3. Pembuatan MahasiswaService

```
1 <?php
2
3 namespace App\Services;
4
5 use App\Repositories\MahasiswaRepository;
6 use App\Repositories\ProdiRepository;
7 use App\Models\Mahasiswa;
8
9 class MahasiswaService
10 {
11     // Constructor Dependency Injection: Service menerima kedua Repository dari luar
12     public function __construct(
13         private MahasiswaRepository $repo,
14         private ProdiRepository $prodiRepo
15     ) {}
16
17     // Controller memanggil ini untuk index(), bukan repo langsung
18     public function list(string $keyword = ''): array
19     {
20         return $this->repo->all($keyword);
21     }
22
23     public function find(int $id): ?Mahasiswa
24     {
25         return $this->repo->find($id);
26     }
27
28     // Dipakai untuk dropdown prodi di form create/edit
29     public function prodiOptions(): array
30     {
31         return $this->prodiRepo->all();
32     }
33
34     public function create(array $input): array
35     {
36         $errors = $this->validate($input);
37         if (empty($errors)) {
38             return ['success' => true, 'errors' => $errors];
39         }
40
41         try {
42             $mhs = new Mahasiswa(
43                 $input['nim'],
44                 $input['nama'],
45                 $input['email'] ?? '',
46                 (int) ($input['prodi_id'] ?? 0),
47                 (int) ($input['angkatan'] ?? date('Y'))
48             );
49             $this->repo->create($mhs);
50             return ['success' => true];
51         } catch (\InvalidArgumentException $e) {
52             return ['success' => false, 'errors' => ['nama' => $e->getMessage()]];
53         }
54     }
55
56     public function update(int $id, array $input): array
57     {
58         // Saat update, NIM milik data ini sendiri tidak dianggap duplikat (lihat $ignoreId)
59         $errors = $this->validate($input, $id);
60         if (empty($errors)) {
61             return ['success' => true, 'errors' => $errors];
62         }
63
64         try {
65             $mhs = new Mahasiswa(
66                 $input['nim'],
67                 $input['nama'],
68                 $input['email'] ?? '',
69                 (int) ($input['prodi_id'] ?? 0),
70                 (int) ($input['angkatan'] ?? date('Y')),
71                 $id
72             );
73             $this->repo->update($mhs);
74             return ['success' => true];
75         } catch (\InvalidArgumentException $e) {
76             return ['success' => false, 'errors' => ['nama' => $e->getMessage()]];
77         }
78     }
79
80     public function delete(int $id): bool
81     {
82         $this->repo->delete($id);
83         return true;
84     }
85
86     // Seluruh logika bisnis/validasi ada di sini, bukan di Controller
87     private function validate(array $input, ?int $ignoreId = null): array
88     {
89         $errors = [];
90
91         if (empty($input['nim'])) {
92             $errors['nim'] = 'NIM wajib diisi.';
93         } elseif (!ctype_digit($input['nim'])) {
94             $errors['nim'] = 'NIM harus berupa angka.';
95         } elseif ($this->repo->existsByNim($input['nim'], $ignoreId)) {
96             $errors['nim'] = 'NIM sudah terdaftar.';
97         }
98
99         if (empty($input['nama'])) {
100             $errors['nama'] = 'Nama wajib diisi.';
101         }
102
103         if (!empty($input['email']) && !filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
104             $errors['email'] = 'Format email tidak valid.';
105         }
106
107         $prodiId = (int) ($input['prodi_id'] ?? 0);
108         if ($prodiId <= 0 || !$this->prodiRepo->exists($prodiId)) {
109             $errors['prodi_id'] = 'Program studi tidak valid.';
110         }
111
112         return $errors;
113     }
114 }
```

MahasiswaService dibuat dengan menerima MahasiswaRepository dan ProdiRepository melalui *constructor injection*. Method `validate()` (bersifat `private`) memeriksa seluruh aturan bisnis: NIM wajib diisi, NIM harus berupa angka, NIM tidak boleh duplikat, nama wajib diisi, format email valid (jika diisi), dan program studi harus tersedia. Method `create()` dan `update()` memanggil `validate()` terlebih dahulu; apabila ditemukan error, keduanya mengembalikan `['success' => false, 'errors' => [...]]` tanpa menyentuh database sama sekali, dan baru memanggil Repository untuk menyimpan data apabila validasi berhasil.

4. Penyederhanaan MahasiswaController

```
1 <?php
2
3 namespace App\Controllers;
4
5 use App\Core\Controller;
6 use App\Services\MahasiswaService;
7
8 class MahasiswaController extends Controller
9 {
10     // Controller hanya menerima Service -> tidak ada lagi validasi atau query di sini
11     public function __construct(private MahasiswaService $service) {}
12
13     public function index(): void
14     {
15         $keyword = trim($_GET['q'] ?? '');
16         $mahasiswa = $this->service->list($keyword);
17
18         $this->view('mahasiswa/index', [
19             'title' => 'Data Mahasiswa',
20             'mahasiswa' => $mahasiswa,
21             'keyword' => $keyword,
22         ]);
23     }
24
25     public function create(): void
26     {
27         $this->view('mahasiswa/create', [
28             'title' => 'Tambah Mahasiswa',
29             'prodiList' => $this->service->prodiOptions(),
30         ]);
31     }
32
33     public function store(): void
34     {
35         // Controller hanya meneruskan request ke Service, lalu menentukan response
36         $result = $this->service->create($_POST);
37
38         if ($result['success']) {
39             $this->flash('success', 'Data mahasiswa berhasil ditambahkan.');
```

MahasiswaController direfactor sehingga hanya menerima MahasiswaService melalui constructor, menggantikan MahasiswaRepository yang disuntikkan langsung pada praktikum sebelumnya. Method store() dan update() pada Controller kini hanya memanggil \$this->service->create(\$_POST) atau \$this->service->update(\$id, \$_POST), memeriksa hasil success, lalu menentukan flash message dan redirect yang sesuai tanpa ada satu baris pun kode validasi atau query database di dalam Controller.

5. Wiring Dependency di public/index.php

```
15 $pdo = App\Core\Database::getInstance();
16 $mahasiswaRepo = new App\Repositories\MahasiswaRepository($pdo);
17 $prodiRepo = new App\Repositories\ProdiRepository($pdo);
18 $mahasiswaService = new App\Services\MahasiswaService($mahasiswaRepo, $prodiRepo);
19 $mahasiswaController = new App\Controllers\MahasiswaController($mahasiswaService);
20
21 // Container: controller yang sudah dirakit lengkap dengan dependency-nya
22 $container = [
23     'MahasiswaController' => $mahasiswaController,
24 ];
```

Proses perakitan dependency diperbarui: Database menghasilkan koneksi PDO, disuntikkan ke MahasiswaRepository dan ProdiRepository, keduanya kemudian disuntikkan ke MahasiswaService, dan terakhir MahasiswaService disuntikkan ke MahasiswaController — mencerminkan alur dependency berlapis yang konsisten dengan prinsip Object Composition pada praktikum sebelumnya.

6. Pengujian Validasi dan Flash Message

Pengujian dilakukan terhadap seluruh skenario validasi melalui form tambah dan edit mahasiswa:

Skenario	Hasil
Tambah data valid	Redirect ke /mahasiswa, flash hijau "Data mahasiswa berhasil ditambahkan."
NIM dikosongkan	Flash merah "NIM wajib diisi."
NIM berisi huruf	Flash merah "NIM harus berupa angka."
NIM duplikat	Flash merah "NIM sudah terdaftar."
Nama dikosongkan	Flash merah "Nama wajib diisi."
Email tidak valid	Flash merah "Format email tidak valid."
Prodi tidak dipilih	Flash merah "Program studi tidak valid."
Edit data tanpa mengubah NIM	Berhasil disimpan (NIM sendiri tidak dianggap duplikat)
Refresh halaman setelah flash muncul	Flash tidak muncul lagi
Hapus data	Flash hijau "Data mahasiswa berhasil dihapus."

SI Akademik Mahasiswa Prodi Mata Kuliah

Data mahasiswa berhasil ditambahkan. ✕

Data Mahasiswa

[+ Tambah Mahasiswa](#)

NIM	Nama	Email	Prodi	Angkatan	Aksi
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus

SI Akademik

Mahasiswa Prodi Mata Kuliah

NIM sudah terdaftar.

Data Mahasiswa

+ Tambah Mahasiswa

Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus

SI Akademik

Mahasiswa Prodi Mata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
230011	Engno	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus

Seluruh pengujian berjalan sesuai yang diharapkan, membuktikan bahwa validasi dan logika bisnis sepenuhnya berpindah ke MahasiswaService, sementara Controller hanya berperan sebagai penghubung antara request dan response.

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan Service Layer berhasil memisahkan logika bisnis, seperti validasi NIM, nama, email, dan program studi, dari *Controller*, sehingga MahasiswaController menjadi lebih ringkas dan berfokus pada pengaturan alur *request-response*. Penerapan Dependency Injection juga tetap konsisten melalui penyuntikan MahasiswaRepository dan ProdiRepository ke dalam MahasiswaService menggunakan *constructor*. Pembagian tanggung jawab antara *Controller* sebagai pengatur alur, *Service* sebagai pengelola logika bisnis, dan *Repository* sebagai pengelola akses database membuat struktur kode lebih terorganisasi, mudah dibaca, diuji, serta dikembangkan. Selain itu, Flash Message berbasis `$_SESSION` berhasil memberikan umpan balik kepada pengguna setelah proses tambah, ubah, maupun hapus data, baik ketika proses berhasil maupun gagal, dan hanya ditampilkan satu kali karena dihapus setelah ditampilkan. Penerapan validasi berlapis pada *Service* dan *entity* Mahasiswa juga membantu menjaga integritas data tanpa menduplikasi tanggung jawab. Dengan demikian, semakin kompleks suatu aplikasi, semakin penting penerapan pemisahan antara alur *request*, logika bisnis, dan akses database agar kode tetap rapi, mudah dipelihara, dan dapat dikembangkan secara berkelanjutan.